



Logistic Regression & Nonlinear Classifiers

Data 442 / 610 Lecture 3

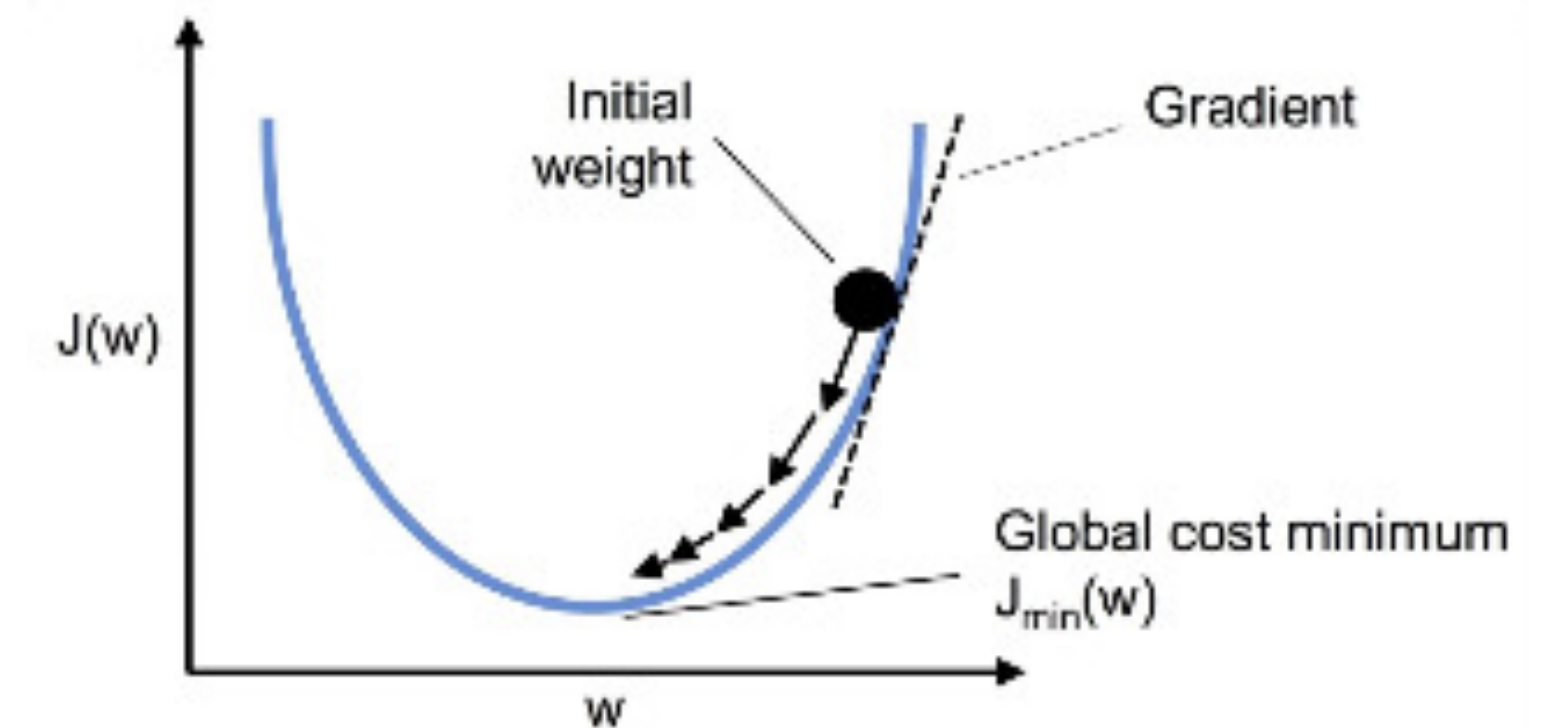
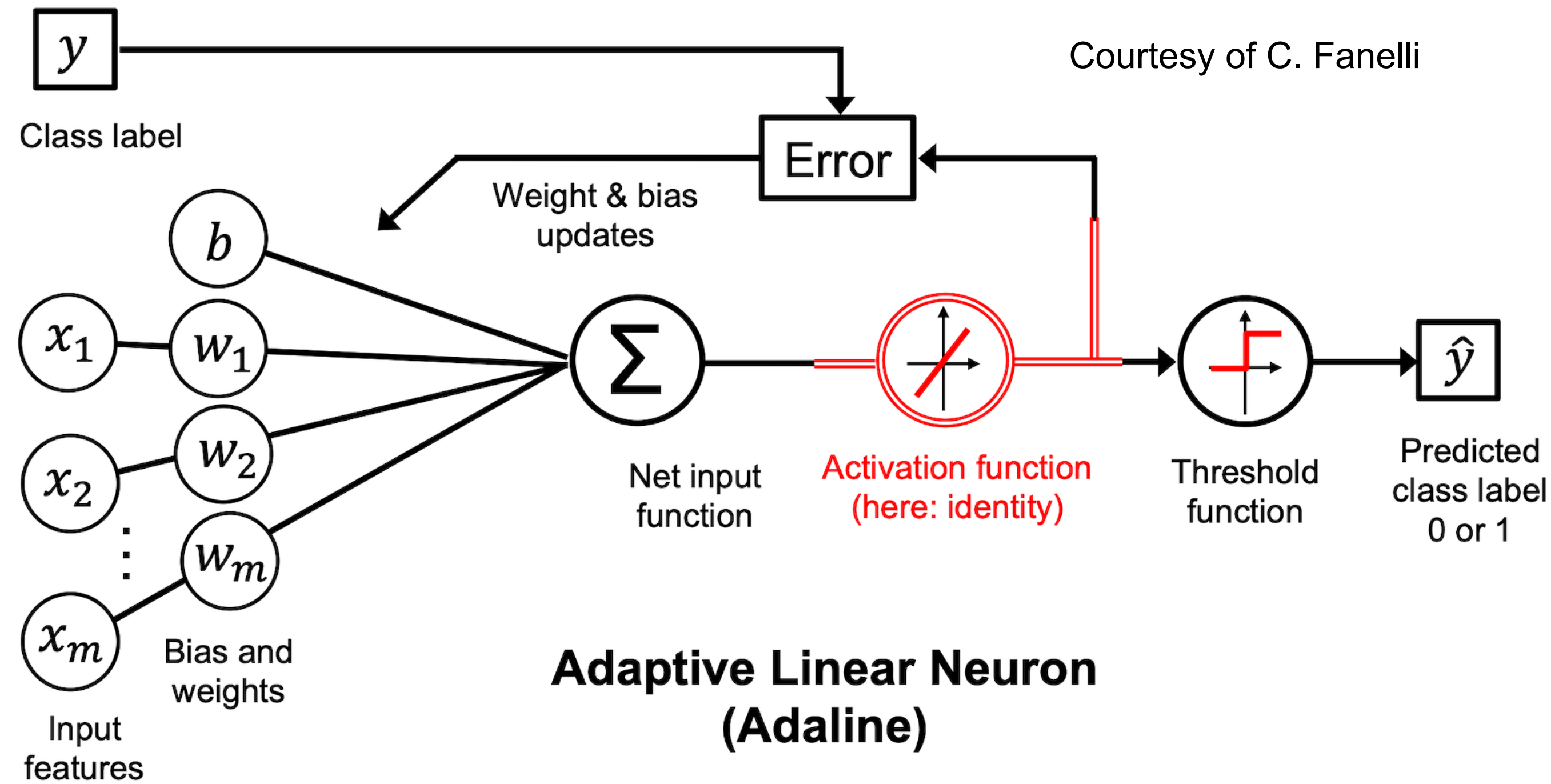
Outline

- Review of Adaline
- Gradient Descent vs. Stochastic Gradient Descent
- Logistic Regression
 - Maximum Likelihood derivation
 - Demo
 - Extensions

Roadmap to Deep Learning



Adaline Review



Mean Squared Error (MSE) Loss

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (y^i - z^i)^2$$

Gradient Descent: $d\mathcal{L} = \vec{\nabla} \mathcal{L} \cdot d\vec{w}$

$d\mathcal{L}$ is most *negative* when $d\vec{w}$ points *opposite* the gradient

Recall $\vec{A} \cdot \vec{B} = AB \cos \theta_{AB}$ and $\cos 180^\circ = -1$

$$\Delta w_i = \frac{2\alpha}{N} \sum_{j=1}^N (y^j - z^j) x_i^j$$

$$\Delta b = \frac{2\alpha}{N} \sum_{j=1}^N (y^j - z^j)$$

Adaline: Takeaways

The good

- Update rule comes from “somewhere”
- Continuous error function allows gradient descent

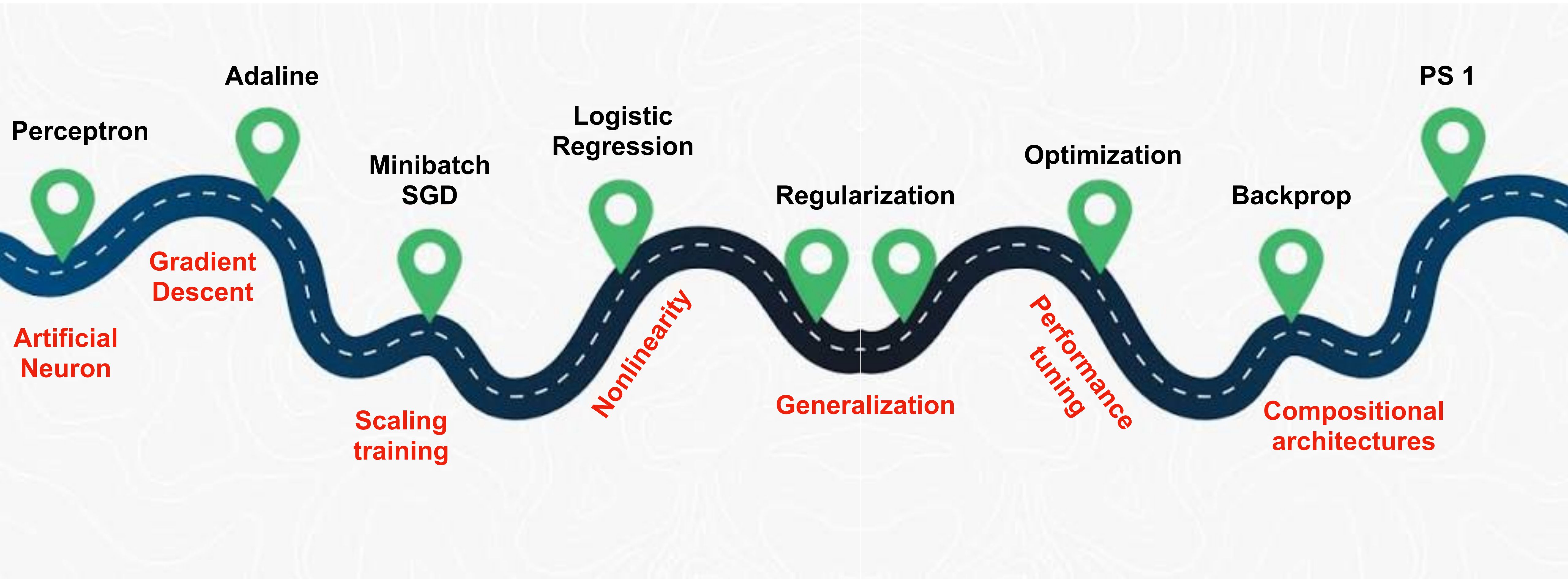
The bad

- Does not allow nonlinear decision boundaries

The ugly

- Learning rate can make or break fitting

Roadmap to Deep Learning



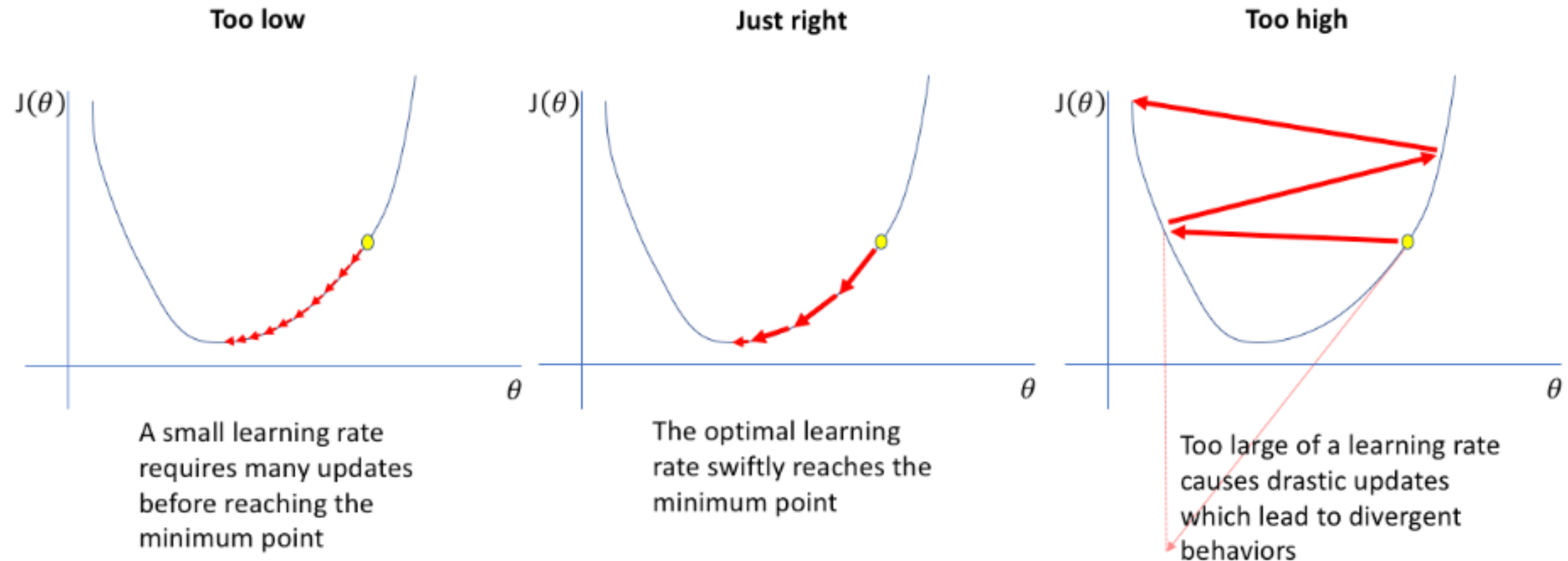
Stochastic Gradient Descent

- **Gradient descent** update rule uses the entire dataset
- **Stochastic gradient descent** updates after each data point
 - *Pros:* faster updates, stochasticity helps avoid local minima
 - *Cons:* unstable convergence, learning rate tuning

Hands-on demo: SGD training loop



The importance of learning rate



Stochastic gradient descent benefits from **learning rate decay**

Courtesy of C. Fanelli

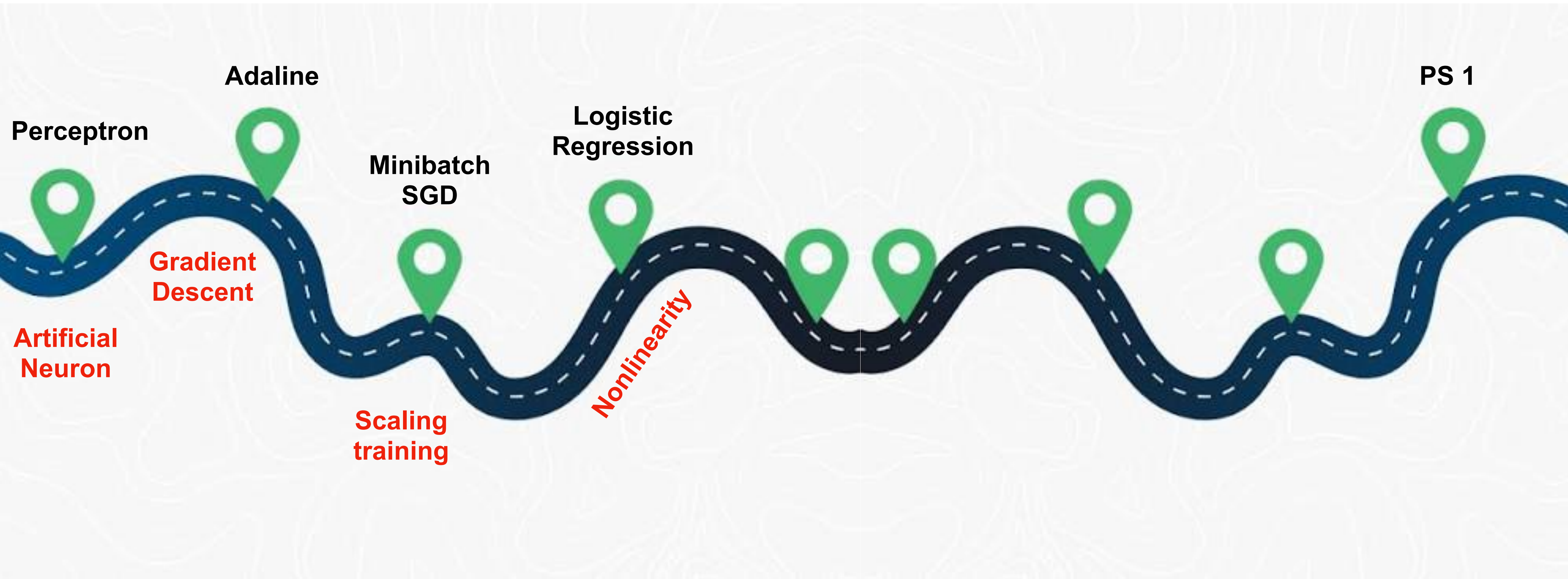
Stochastic Gradient Descent

- **Gradient descent** update rule uses the entire dataset
- **Stochastic gradient descent** updates after each data point
 - *Pros:* faster updates, stochasticity helps avoid local minima
 - *Cons:* unstable convergence, learning rate tuning
- **Mini-batch gradient descent** updates after each subset of data points
 - Improved gradient estimate, benefits from parallelism

Hands-on demo: Mini-batch SGD training loop



Roadmap to Deep Learning



Logistic Regression

- Adaline uses a continuous objective $\mathcal{L}$ and linear activation $\sigma(z) = z$
- Logistic regression uses a nonlinear activation to predict class probabilities
- The **preactivation** is a logit (log-odds)

$$z \equiv \text{logit}(p) = \log \frac{p}{1-p}$$

Preactivation: $z = \mathbf{w}^\top \mathbf{x} + b$

Sigmoid: $\sigma(z) = \frac{1}{1 + e^{-z}}$

$$\sigma(z) : (-\infty, \infty) \rightarrow (0, 1)$$

Logistic Regression: Loss function

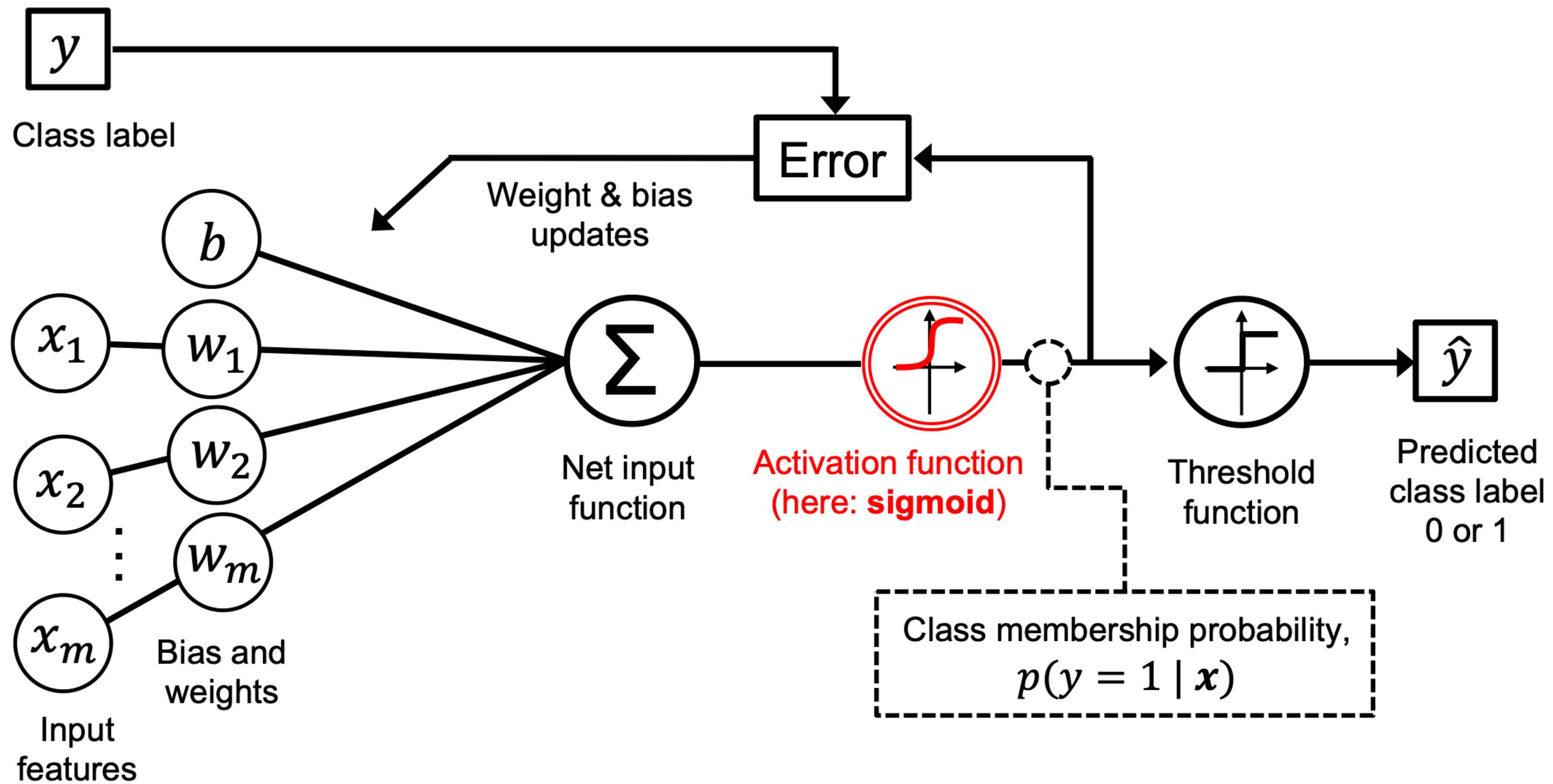
Goal: Find $\mathbf{w}$ to maximize the *likelihood* of observations $\mathbf{w}^* = \operatorname{argmax}_{\mathbf{w}} \left[p(y | \mathbf{x}; \mathbf{w}, b) \right]$ **Maximum Likelihood Estimation**

$$p(y | \mathbf{x}; \mathbf{w}, b) \sim \prod_i p(y^i | \mathbf{x}^i; \mathbf{w}, b) = \prod_i \sigma(z^i)^{y^i} \cdot (1 - \sigma(z^i))^{1-y^i} \quad \text{Bernoulli distribution}$$

$$\mathcal{L} = \sum_i -y^i \log \sigma(z^i) - (1 - y^i) \log (1 - \sigma(z^i)) \quad \text{Cross entropy loss}$$

Goal: Find $\mathbf{w}$ to minimize the *cross entropy* loss $\mathcal{L} = \sum_i -y^i \log \sigma(z^i) - (1 - y^i) \log (1 - \sigma(z^i))$

Logistic Regression



Hands-on Demo: Logistic Regression



Multiclass Regression: Softmax

- Logistic regression extends to multiple classes

$$z_j = \sum_k w_{jk} \cdot x_k + b_k$$

- Each preactivation is an un-normalized log-probability

$$z_j \equiv \log \tilde{p}$$

- The corresponding activation is the **softmax** function

$$\text{softmax } z_j = \frac{\exp z_j}{\sum_k \exp z_k}$$

Numerically-stable softmax

$$\text{softmax } (z_j - \max \mathbf{z}) = \text{softmax } z_j$$

Views z_j as a log-odds with respect to the most-probable outcome

$$z_j \sim \log \frac{p_j}{p_{\max}}$$

That's all!

- Slides will be posted on the course website: <https://colen-teaching.github.io/DATA442-Fall-2026>
- Lab demos will also be posted
- Be on the lookout for PS 0: A **Bonus** Assignment
- Have a great Labor Day Weekend!